DÉTECTEZ DES FAUX BILLETS AVEC PYTHON

MK

EXECUTIVE SUMMARY

A – Préparation des données

Le fichier contient des données manquantes

Nous imputons ces valeurs avec KNNImputer et un nombre de voisins optimal de 10

Les marges et la longueur sont les variables les plus différenciantes

La projection PCA montre que la preuve de la véracité des billets n'est pas évidente

B – Algorithmes de classification

K-Means performe bien sur les Faux Positifs, et moins bien sur les Faux Négatifs

La projection T-SNE met en évidence les erreurs potentielles de K-Means

La Régression Logistique performe bien sur l'ensemble des métriques

Le Classifieur K-Neighbors performe bien sur les Faux Négatifs et moins bien sur les Faux Positifs

Le Classifieur Random Forest performe bien sur les Faux Négatifs et moins bien sur les Faux Positifs

C – Choix et amélioration du modèle final

La Régression Logistique est plus adaptée à nos données et nos objectifs

Notre modèle final : une Régression Logistique optimisée avec GridSearchCV

D – Conclusion

A - PRÉPARATION DES DONNÉES

Le fichier contient des données manquantes

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   is_genuine    1500 non-null   bool
 1   diagonal      1500 non-null   float64
 2   height_left   1500 non-null   float64
 3   height_right  1500 non-null   float64
 4   margin_low    1463 non-null   float64
 5   margin_up     1500 non-null   float64
 6   length        1500 non-null   float64
dtypes: bool(1), float64(6)
memory usage: 71.9 KB
```

is_genuine	count
True	1000
False	500

dtype: int64

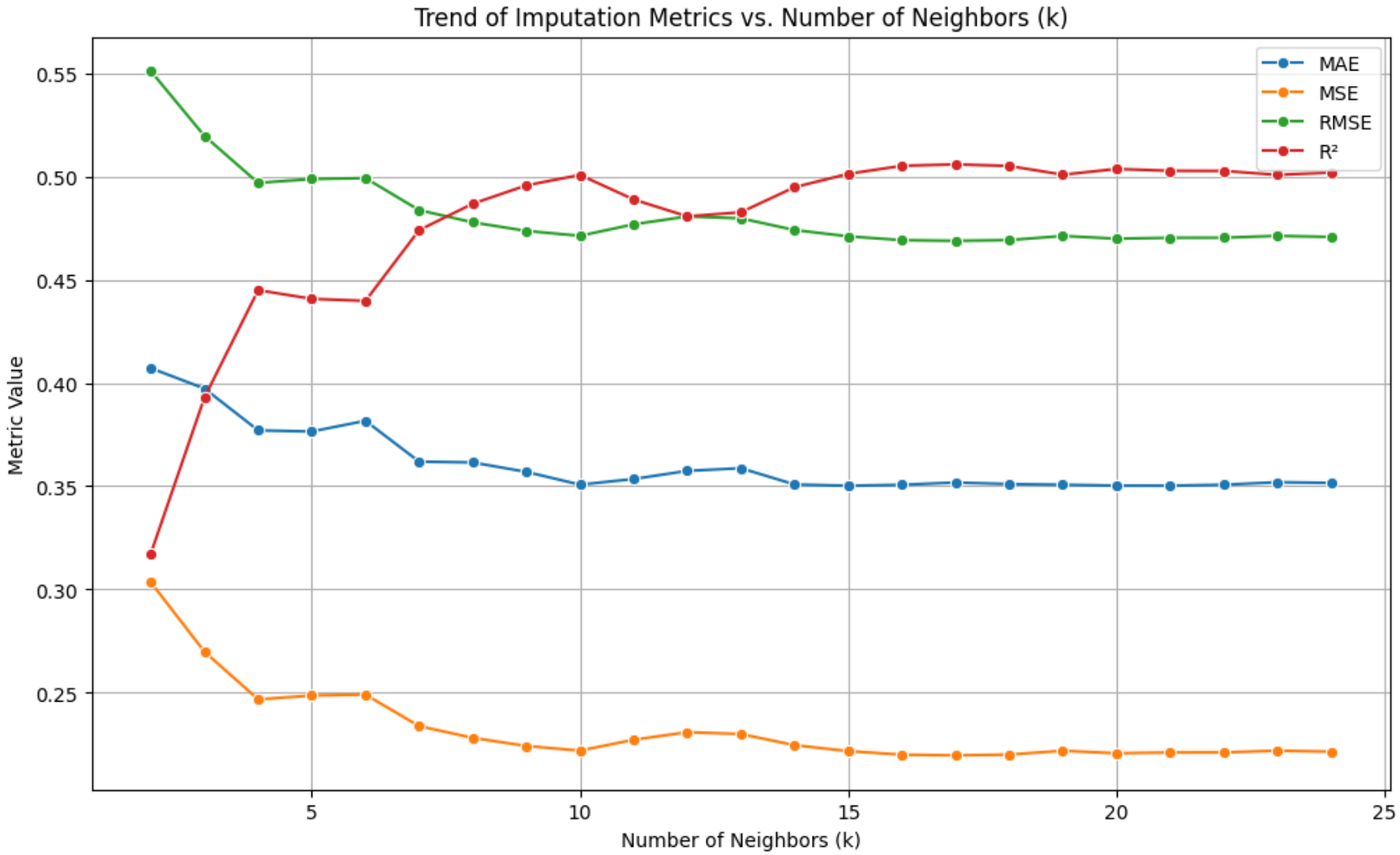
Pourcentage de données manquantes :

	0
margin_low	2.47

dtype: float64

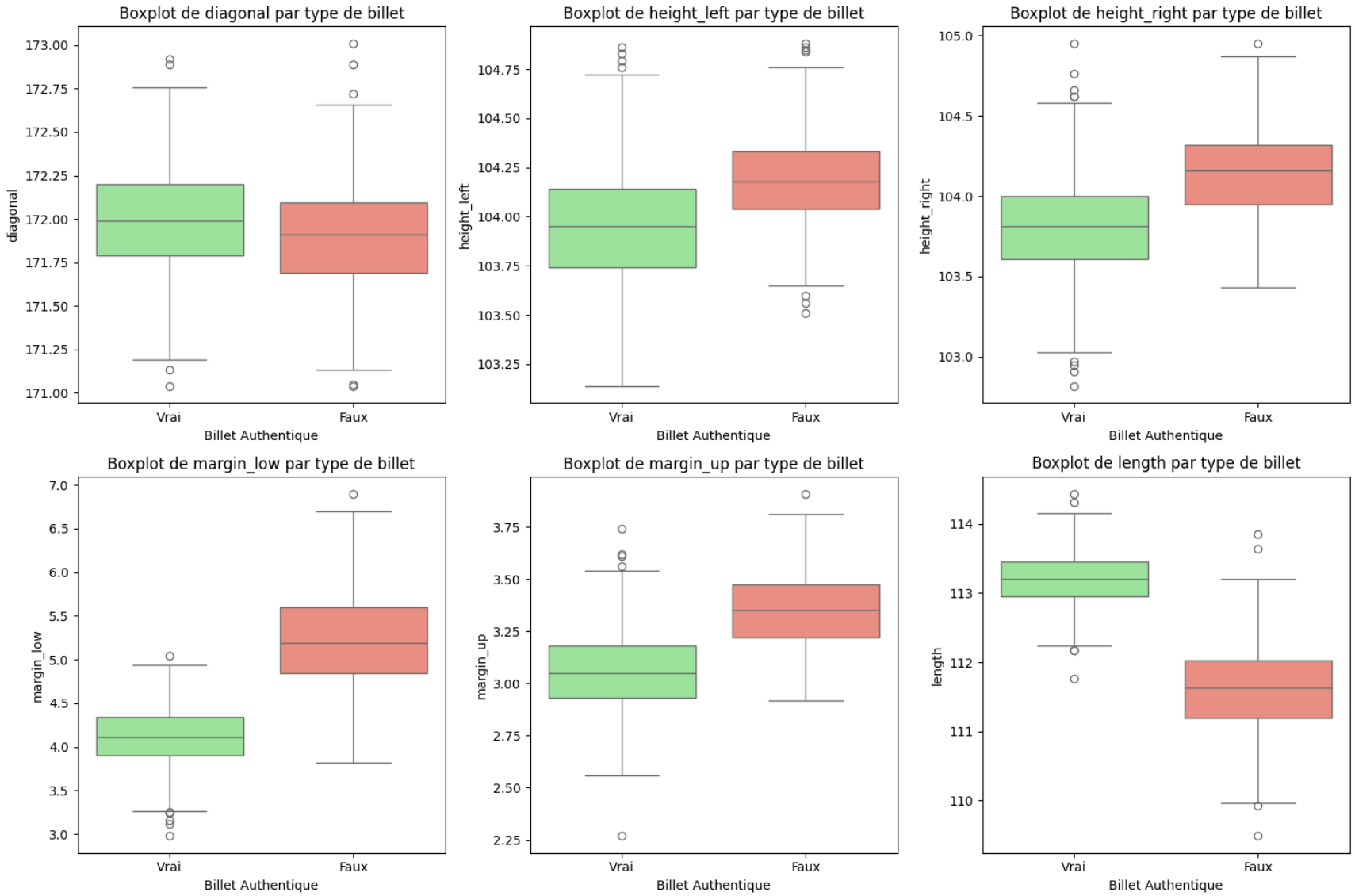
A - PRÉPARATION DES DONNÉES

Nous imputons ces valeurs avec KNNImputer et un nombre de voisins optimal de 10



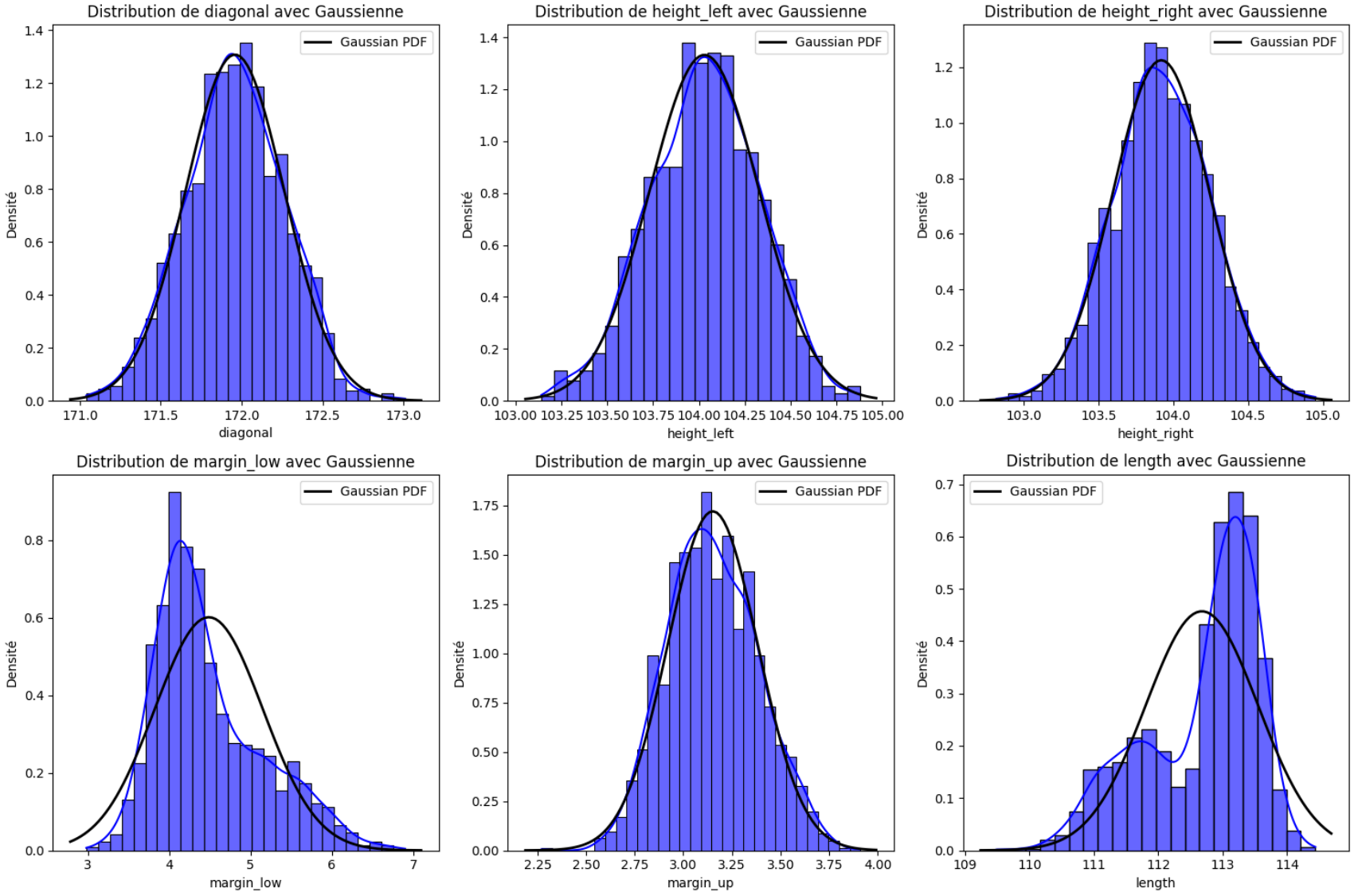
A - PRÉPARATION DES DONNÉES

Les marges et la longueur sont les variables les plus différenciantes



A - PRÉPARATION DES DONNÉES

Les marges et la longueur sont les variables les plus différenciantes



A - PRÉPARATION DES DONNÉES

Les marges et la longueur sont les variables les plus différenciantes

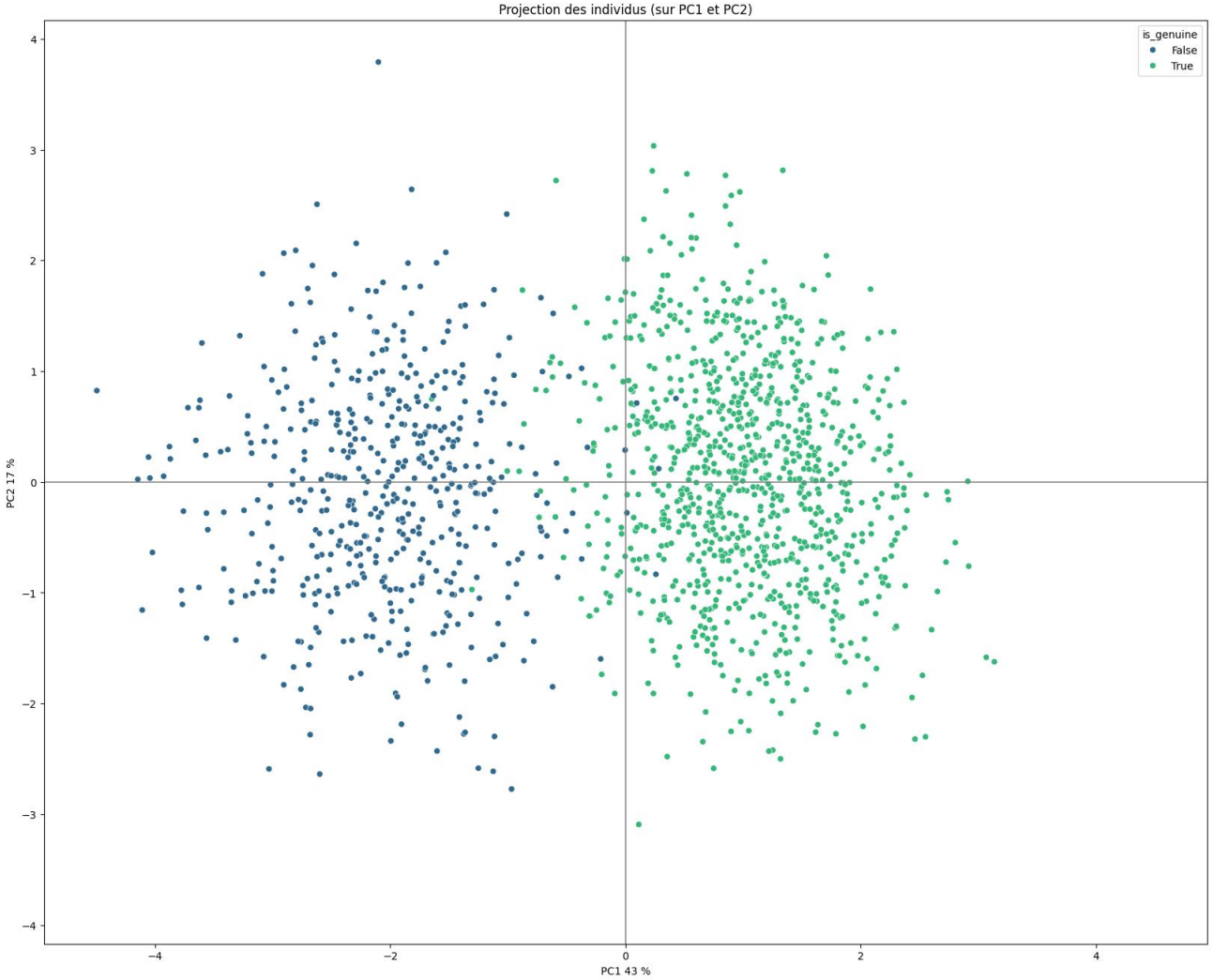
Tableau de comparaison des individus moyens

	diagonal	height_left	height_right	margin_low	margin_up	length
False	171.901160	104.190340	104.14362	5.215935	3.350160	111.630640
True	171.987080	103.949130	103.80865	4.116097	3.052130	113.202430
Pourcentage d'écart	-0.049957	0.232046	0.32268	26.720415	9.764656	-1.388477

A - PRÉPARATION DES DONNÉES

La projection PCA montre que la preuve de la véracité des billets n'est pas évidente

Projection PCA

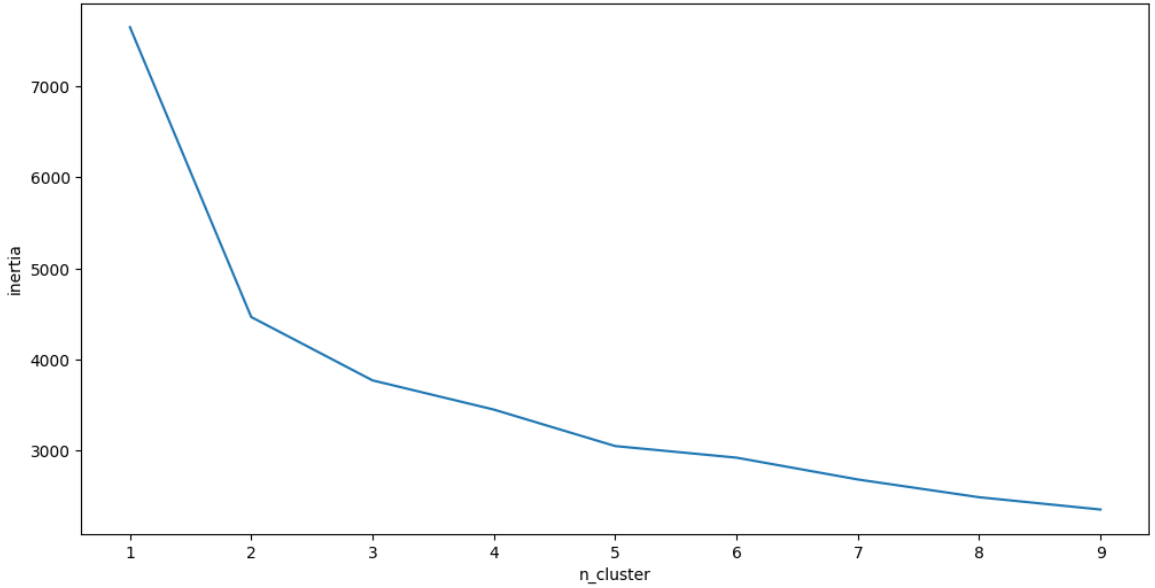


B – ALGORITHMES DE CLASSIFICATION

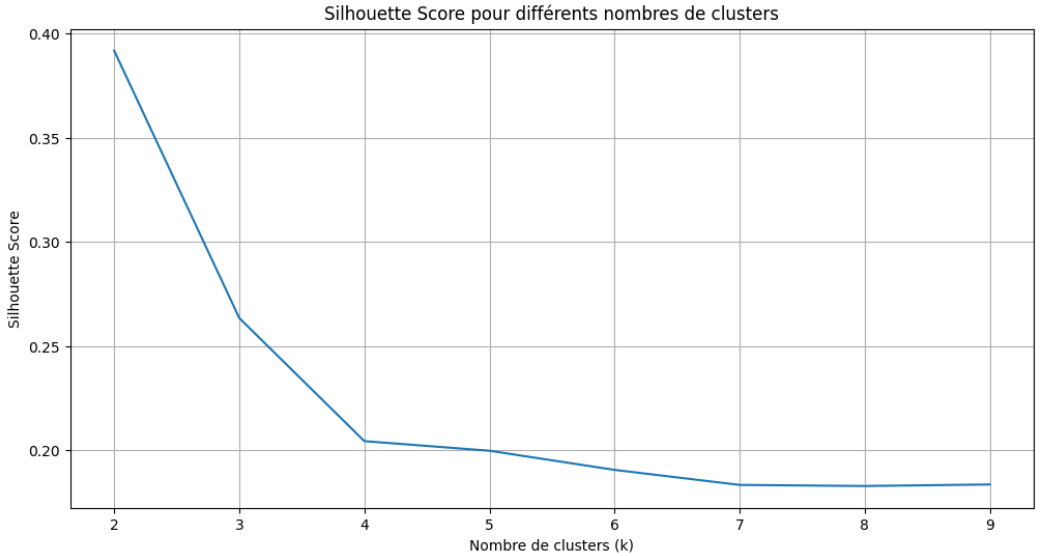
K-Means performe bien sur les Faux Positifs, et moins bien sur les Faux Négatifs

Attention : Clustering ≠ Classification
Testons tout de même Kmeans sur nos données

Méthode du coude



Silhouette score

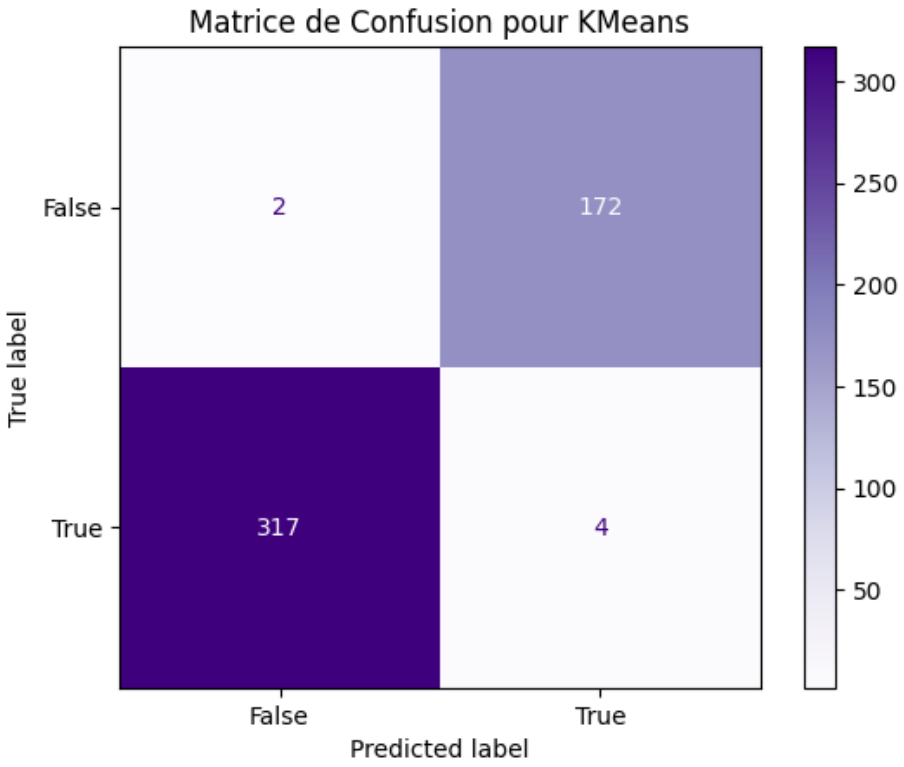


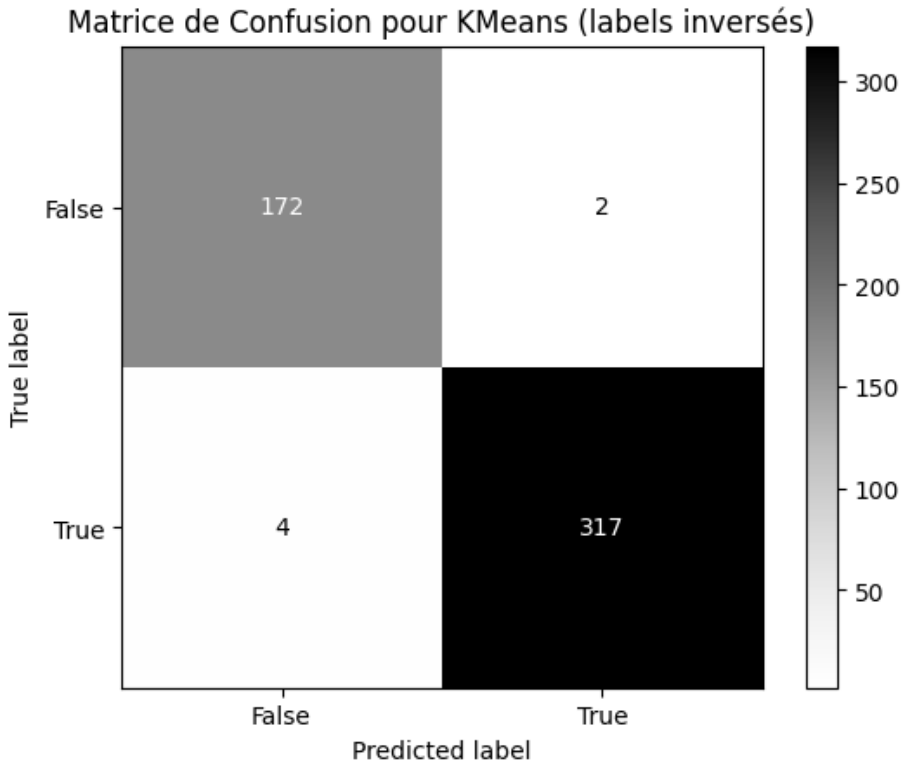
Les deux méthodes indiquent un nombre de clusters optimal de 2
C'est cohérent avec nos objectifs

B – ALGORITHMES DE CLASSIFICATION

K-Means performe bien sur les Faux Positifs, et moins bien sur les Faux Négatifs

Attention : Clustering ≠ Classification



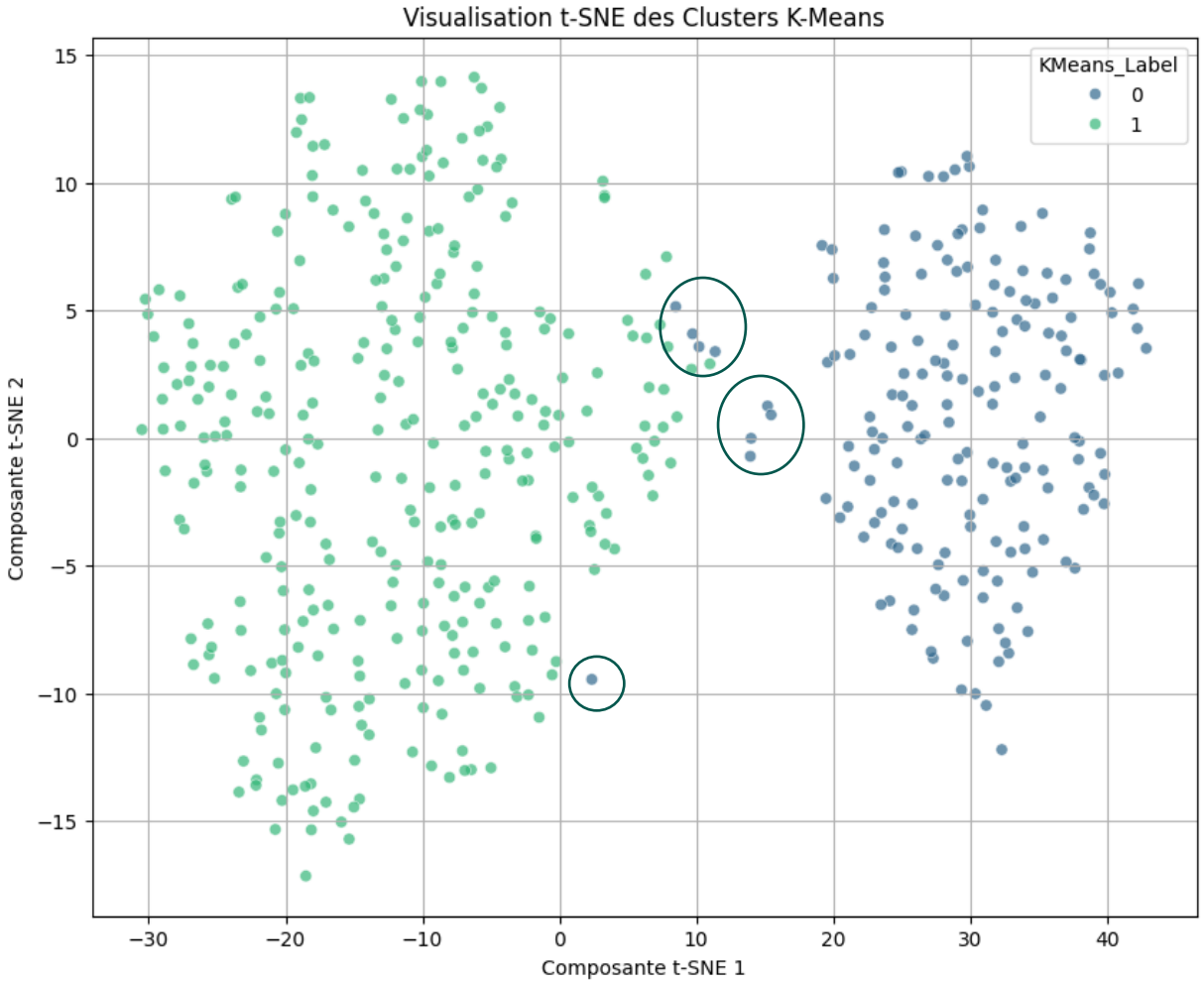



```
Accuracy du classifieur K-Means (labels inversés) : 0.9878787878787879
Precision du classifieur K-Means (labels inversés) : 0.9937304075235109
Recall du classifieur K-Means (labels inversés) : 0.9875389408099688
F1 score du classifieur K-Means (labels inversés) : 0.990625
ROC-AUC score du classifieur K-Means (labels inversés) : 0.9880223439682028
```

B – ALGORITHMES DE CLASSIFICATION

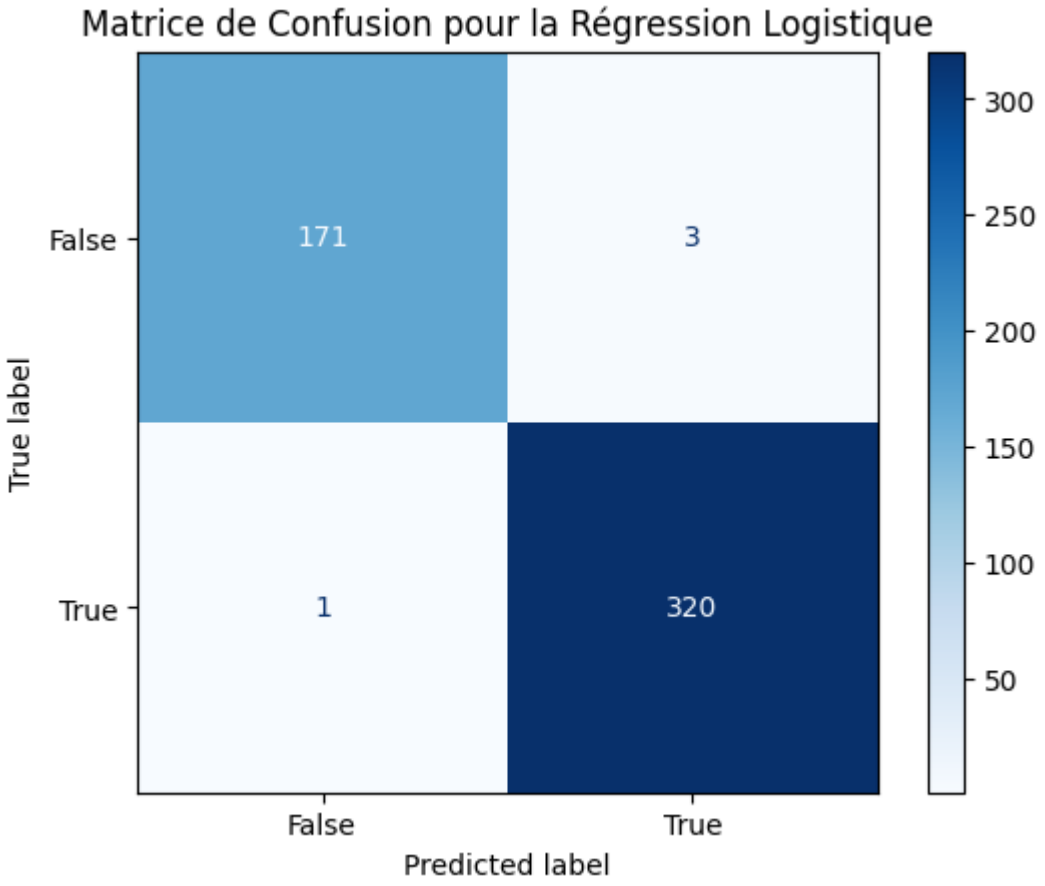
La projection T-SNE met en évidence les erreurs potentielles de K-Means

Projection t-SNE



B – ALGORITHMES DE CLASSIFICATION

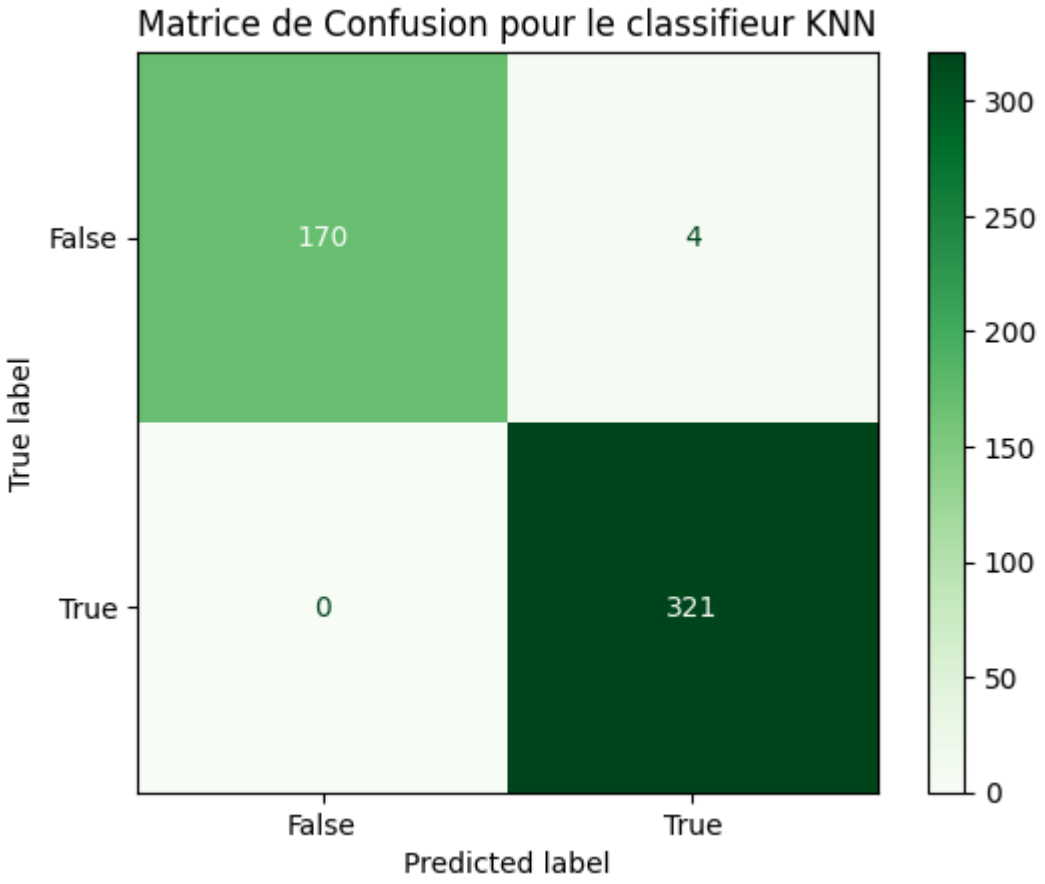
La Régression logistique performe bien sur l'ensemble des métriques




```
Accuracy de la régression logistique : 0.9919191919191919
Precision de la régression logistique : 0.9907120743034056
Recall de la régression logistique : 0.9968847352024922
F1 score de la régression logistique : 0.9937888198757764
ROC-AUC score de la régression logistique : 0.9898216779460738
```

B – ALGORITHMES DE CLASSIFICATION

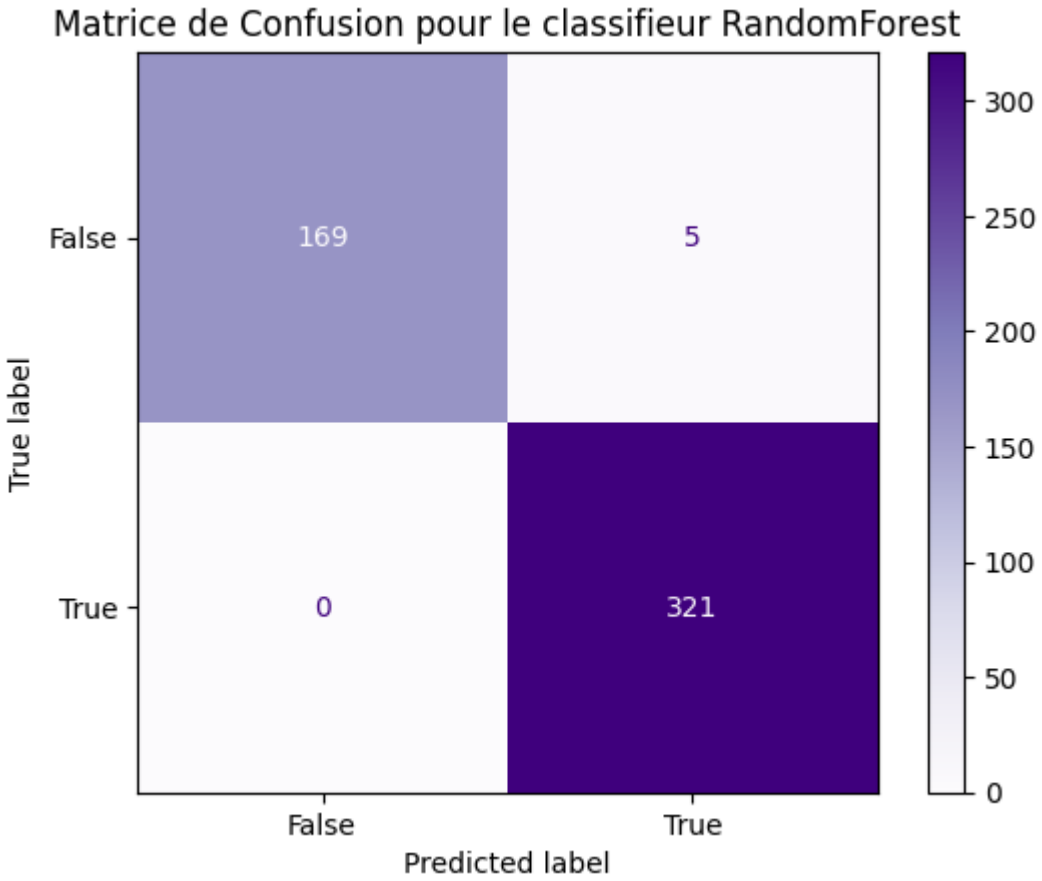
Le Classifieur K-Neighbors performe bien sur les Faux Négatifs et moins bien sur les Faux Positifs




```
Accuracy du classifieur KNN : 0.9919191919191919
Precision du classifieur KNN : 0.9876923076923076
Recall du classifieur KNN : 1.0
F1 score du classifieur KNN : 0.9938080495356038
ROC-AUC score du classifieur KNN : 0.9885057471264368
```

B – ALGORITHMES DE CLASSIFICATION

Le Classifieur Random Forest performe bien sur les Faux Négatifs et moins bien sur les Faux Positifs




```
Accuracy du classifieur Random Forest : 0.98989898989899
Precision du classifieur Random Forest : 0.9846625766871165
Recall du classifieur Random Forest : 1.0
F1 score du classifieur Random Forest : 0.9922720247295209
ROC-AUC score du classifieur Random Forest : 0.985632183908046
```

C – CHOIX ET AMÉLIORATION DU MODÈLE FINAL

La Régression Logistique est plus adaptée à nos données et nos objectifs

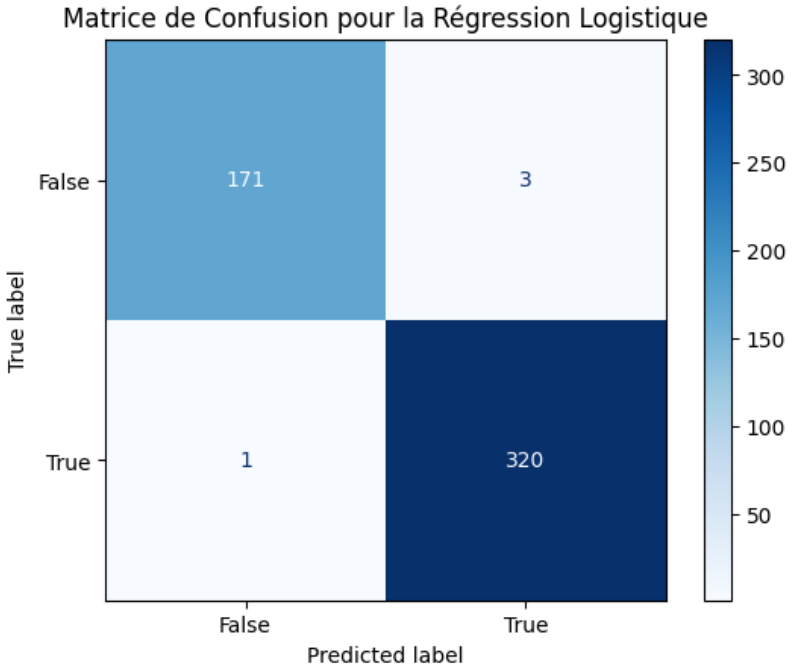
	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	0.9919	0.9907	0.9969	0.9938	0.9898
1	K-Neighbors	0.9919	0.9877	1.0000	0.9938	0.9885
2	Random Forest	0.9899	0.9847	1.0000	0.9923	0.9856
3	K-Means (mapped labels)	0.9879	0.9937	0.9875	0.9906	0.9880

Afin de limiter les Faux Positifs : privilégions Precision

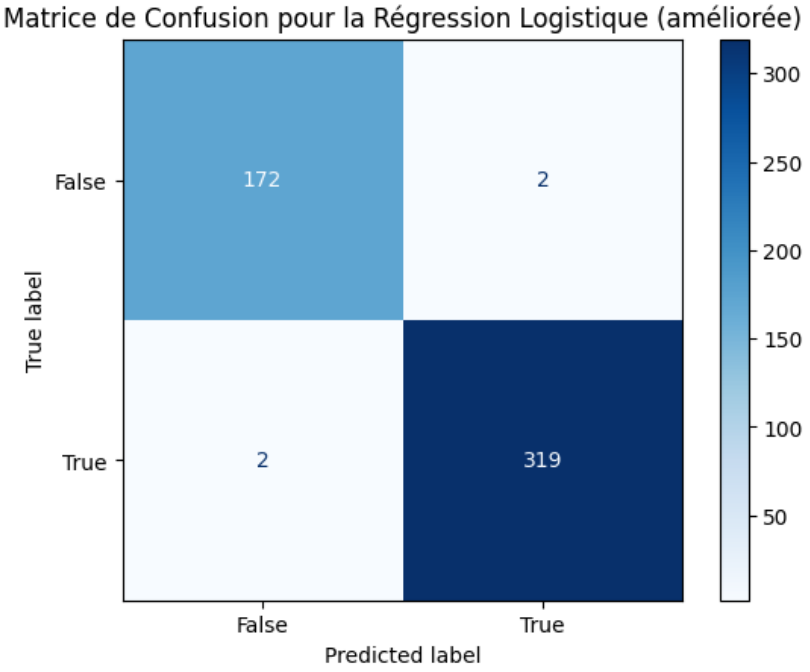
C – CHOIX ET AMÉLIORATION DU MODÈLE FINAL

Notre modèle final : une Régression Logistique optimisée via GridSearchCV

```
Accuracy de la régression logistique : 0.9919191919191919
Precision de la régression logistique : 0.9907120743034056
Recall de la régression logistique : 0.9968847352024922
F1 score de la régression logistique : 0.9937888198757764
ROC-AUC score de la régression logistique : 0.9898216779460738
```

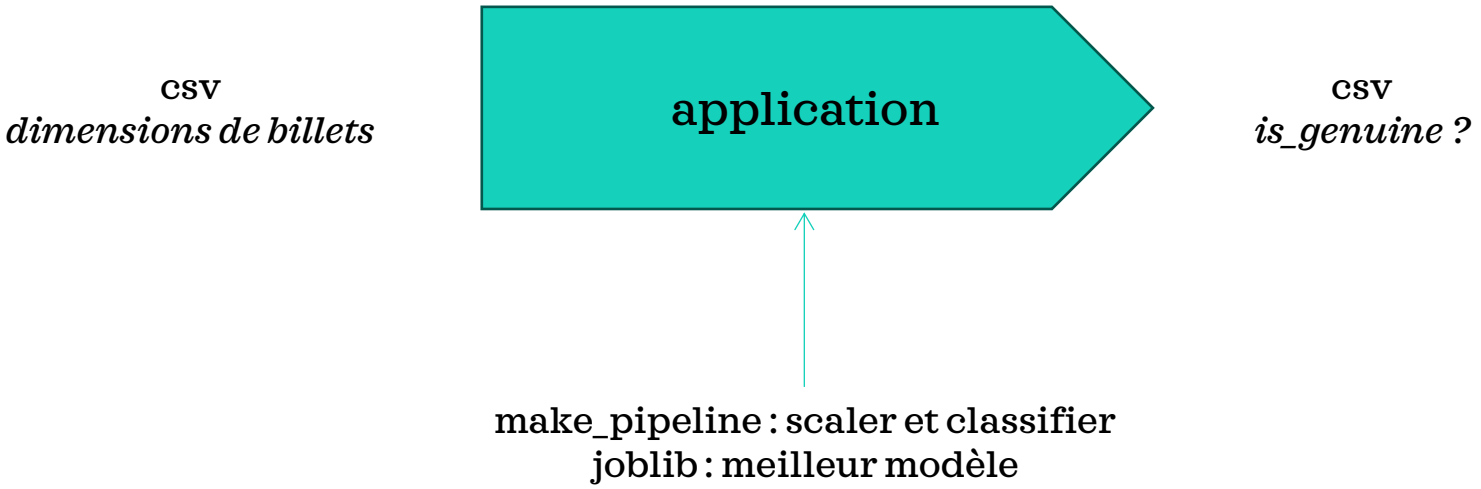



```
Accuracy de la régression logistique améliorée : 0.9919
Precision de la régression logistique améliorée : 0.9938
Recall de la régression logistique améliorée : 0.9938
F1 score de la régression logistique améliorée : 0.9938
ROC-AUC score de la régression logistique améliorée : 0.9911
```



C – CHOIX ET AMÉLIORATION DU MODÈLE FINAL

Notre modèle final : une Régression Logistique optimisée via GridSearchCV



Commande pour lancer l'application dans le terminal :

```
python votre_script.py --csv_path
chemin/vers/votre_fichier.csv
--model_path chemin/vers/votre_modele.joblib
```

D – CONCLUSION

Nous avons nettoyé le fichier et utilisé KNNImputer pour les valeurs manquantes.

Ensuite, nous avons identifié la Régression Logistique
comme la modélisation la plus adaptée.

Nous avons optimisé notre modèle de Régression Logistique grâce à GridSearchCV.

Enfin, ce modèle a servi à créer un script exécutable
pour tester de nouveaux échantillons.

Rappel de la commande pour tester les faux billets à exécuter dans le terminal :

```
python votre_script.py --csv_path chemin/vers/votre_fichier.csv
--model_path chemin/vers/votre_modele.joblib
```